Visual Recognition Using Deep Learning Homework 2 Report

Pao-Hua Hsu

National Yang Ming Chiao Tung University, Hsinchu, Taiwan
Student ID: 314581025
paohuah.ii14@nycu.edu.tw
GitHub: https://github.com/benhsu0828/NYCU_VRDL_HW2.git

1 Introduction

This report follows the ECCV-style citation format [2] and summarizes my implementation for the NYCU Visual Recognition Using Deep Learning homework [5]. The task is digit object detection, and my experiments focus on two DETR-style directions: a custom baseline DETR [1] and DN-Deformable-DETR with denoising training [4].

On the model side, I evaluate a baseline DETR pipeline (including FPN-style feature fusion variants) and a DN-DETR pipeline with denoising training. On the data side, I test both the original training set and an offline canvas-augmented version (random paste on enlarged canvas with Gaussian-noise background), then analyze whether this augmentation helps both models consistently.

From the current experiments, the most stable setup is still the baseline DETR family. DN-DETR can be sensitive to preprocessing and hyperparameters, and the canvas-style strong augmentation does not consistently improve final validation performance even when it can sometimes improve convergence behavior. Therefore, this report emphasizes reproducible comparisons, key ablations (with/without offline canvas augmentation), and practical lessons on preprocessing consistency and training stability.

2 Method

2.1 Data Processing and Augmentation

For data processing, the later stage of my work focused mainly on improving DN-DETR. The baseline uses a milder augmentation strategy, mostly photometric and light geometric perturbations, including brightness/contrast jitter and small random scaling and translation under a fixed input size. I also keep class-balanced supervision to make classification training more stable.

For DN-DETR, I tested a canvas expansion + Gaussian noise augmentation strategy: randomly pasting the original image onto a larger canvas (with a fixed scale factor) and filling the background with Gaussian noise (both offline and online versions were tested). The result was that when the canvas scale was larger

than 1.1×, the model almost could not learn effectively. After reducing the scale to at most 1.1×, training could converge, but even after 100 epochs, the final mAP was still not better than the setting without this augmentation. So in the final main DN-DETR setup, I did not use canvas expansion + Gaussian noise, and kept preprocessing consistent with the baseline.

During later debugging, I found that one major bottleneck of DN-DETR was not only from loss or matching settings, but also from image preprocessing. At first, I resized and padded all images to a fixed 720×720 square input. This was a straightforward way to unify data format, but it made digits in originally small images become much larger in pixels, so each query had a harder time telling whether a region was a digit or background. Also, many images are small, so enlarging them to match larger ones significantly increased computation and slowed training. Under this setup, DN-DETR kept a high class_error for a long time, and classification did not learn stably.

Therefore, I changed the DN-DETR train/valid/test pipeline to batch-wise dynamic padding: keep each image at its original size (no resizing), then pad images in the same batch to that batch's maximum height and width, instead of forcing every image to 720×720. This reduces a large amount of invalid background and improves early training stability.

2.2 Model Variants

This implementation mainly compares two directions: a custom baseline DETR and a later attempt to introduce DN-DETR. For both directions, the main backbone is ResNet-50 [3].

In the baseline design, I first tried flattening multiple fused feature levels into transformer encoder tokens, so decoder queries could access high-resolution details and low-resolution semantics at the same time. To preserve scale information, each level feature is combined with its positional embedding and a learnable level embedding before concatenation into the encoder. This gives richer representation, but the transformer needs full attention over many cross-scale tokens, which clearly increases training time.

So the later baseline switched to an FPN-style fusion as the main version: first align channels with 1×1 convolution, then upsample deeper features and add them to shallower features, and finally apply 3×3 convolution to smooth the fused result. Only the highest-resolution fused feature map is sent to the encoder. Compared with the full multi-scale token flattening approach above, this version keeps multi-scale information while significantly reducing computation, with better training efficiency and overall stability.

DN-DETR (DeNoising DETR) [4] is an improved DETR variant that adds denoising training. Original DETR [1] learns end-to-end detection through bipartite matching, but it often has slow convergence and sparse positive supervision in early training. The key idea of DN-DETR is to create an extra set of noisy known targets during training (including label and box perturbations), and train the model to solve both the standard object-query detection task and a denoising reconstruction task. This denoising branch provides denser and more

Table 1: Training hyperparameter comparison between baseline DETR and DN-DETR.

Hyperparameter	Baseline DETR	DN-DETR
Batch size	32	28
Epochs	80	70
Optimizer	AdamW	AdamW
LR / backbone LR	1×10^{-4} / 1×10^{-5}	1×10^{-4} / 1×10^{-5}
Weight decay	1×10^{-4}	1×10^{-4}
LR scheduler	StepLR (`lr_drop` = 70, $\gamma = 0.1$)	ReduceLROnPlateau ($\gamma = 0.5$, patience = 6, threshold = 5×10^{-4})
Gradient clip	0.1	0.1
Num. queries	300	300
Classification loss settings	`ce_loss_coef` = 2.0, `eos_coef` = 0.1, label smoothing = 0.05	`cls_loss_coef` = 2.0, `eos_coef` = 0.05, `-no-class-balance`
DN settings	N/A	scalar = 5, label noise = 0.2, box noise = 0.4
Data augmentation	color jitter only (no canvas/noise)	canvas scale = 1.05, Gaussian noise std = 0.05
Evaluation frequency	disabled during training (no `-eval-every-epoch`)	every 3 epochs (`-eval-interval 3`)

stable learning signals, reduces early matching instability, and usually improves convergence speed and training stability.

Compared with baseline DETR, DN-DETR still keeps a transformer-based end-to-end detection pipeline, but uses stronger training objectives and supervision design. The main differences are: 1) additional denoising queries; 2) auxiliary losses for label/box noise; and 3) more sensitive hyperparameters (such as denoising query count and noise scale). So in practice, besides backbone and decoder design, DN-DETR also relies more on coordinated tuning of preprocessing and loss weights.

2.3 Hyperparameters

The training settings actually used for baseline and DN-DETR are summarized in Tab. 1. Both use AdamW, a two-level learning rate setup (smaller LR for the backbone), and 300 object queries. The main differences are the LR scheduler, DN-related denoising parameters, and data augmentation strategy.

In addition, the DN-DETR version fixes several third-party framework arguments inside `parse_args()`, including `dataset_file=custom_coco`, `modelname=dn_dab_deformable_detr`, `batch_norm_type=FrozenBatchNorm2d`, and `num_select=num_queries`. Also, `--use-dn` is enabled by default in the current code, so all DN-DETR experiments in this report were trained in DN mode.

3 Results

The experimental results are summarized in Tab. 2. I omit most intermediate figures here and keep only the validation scores for clearer comparison. The Baseline with FPN version is with batch max padding but the Baseline with No FPN and No Canvas Offline Aug didn't.

Table 2: Validation score comparison for baseline variants (with/without FPN) and DN-DETR under different offline canvas augmentation settings.

Model / Version	Backbone / Setting	Description	Score
`Baseline (No FPN, No Canvas Offline Aug)`	DETR + ResNet-50	trained on original train set	0.28
`Baseline with FPN (No Canvas Offline Aug)`	DETR + ResNet-50 + FPN fusion	trained on original train set	0
`DN-DETR (No Canvas Offline Aug)`	DN-Deformable-DETR-R50	trained on original train set	0.37
`DN-DETR (Canvas Offline Aug)`	DN-Deformable-DETR-R50	trained on offline canvas-augmented train set	0.26

References

1. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. ArXiv **abs/2005.12872** (2020), `https://api.semanticscholar.org/CorpusID:218889832`
2. European Conference on Computer Vision (ECCV): Author guidelines for eccv submission. `https://eccv.ecva.net/` (2026)
3. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR) pp. 770–778 (2015), `https://api.semanticscholar.org/CorpusID:206594692`
4. Li, F., Zhang, H., Liu, S., Guo, J., shuan Ni, L.M., Zhang, L.: Dn-detr: Accelerate detr training by introducing query denoising. 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 13609–13617 (2022), `https://api.semanticscholar.org/CorpusID:247218212`
5. Lin, Y.Y.: Selected topics in visual recognition using deep learning. `https://docs.google.com/presentation/d/1e8EfifSiTjwda5n1LYNN-dTmR3DSKrA4GPOqhEY2DrQ/edit?usp=sharing` (2026), national Yang Ming Chiao Tung University course slides, Spring 2026